

媒体模块 (Media) 详细报告

一、需求与设计

1.1 功能需求

需求编号	需求描述	优先级
MEDIA-01	文件上传 (图片/视频/音频/头像/背景)	P0
MEDIA-02	文件类型校验 (Content-Type白名单)	P0
MEDIA-03	文件大小限制 (按 UploadKind 区分)	P0
MEDIA-04	静态文件服务 (HTTP Range支持)	P0
MEDIA-05	上传URL动态可配 (多域名迁移)	P1

1.2 设计决策

1. 两段式大小校验

— 先检查客户端报告大小 (允许-1未知大小), 再检查磁盘实际大小 (严格)
2. UUID文件名

— 防止路径遍历和文件名冲突
3. Content-Type白名单

— 按 UploadKind 限制允许的MIME类型
4. HTML Range 支持

— UploadStaticServlet 支持 Range: bytes=START-END 实现断点续传/视频拖动
5. URL前缀动态替换

— MediaUrlBuilder.normalize() 允许切换域名后重写URL前缀

二、后端设计

2.1 核心文件清单

文件	行数	位置
MediaResource.java	37	src/main/java/com/example/chat/media/MediaResource.java
MediaService.java	72	src/main/java/com/example/chat/media/MediaService.java
MediaDao.java	26	src/main/java/com/example/chat/media/MediaDao.java
UploadStaticServlet.java	106	src/main/java/com/example/chat/media/UploadStaticServlet.java
MediaUrlBuilder.java	41	src/main/java/com/example/chat/media/MediaUrlBuilder.java
UploadKind.java	25	src/main/java/com/example/chat/media/UploadKind.java
UploadSizePolicy.java	15	src/main/java/com/example/chat/media/UploadSizePolicy.java
MediaContentTypePolicy.java	26	src/main/java/com/example/chat/media/MediaContentTypePolicy.java

2.2 REST API 端点

HTTP	路径	Content-Type	功能
POST	/api/media	multipart/form-data	上传文件

HTTP	路径	Content-Type	功能
GET	/uploads/{kind}/{uuid.ext}	(按扩展名)	静态文件服务

代码位置：

- REST：src/main/java/com/example/chat/media/MediaResource.java
- Servlet：src/main/java/com/example/chat/media/UploadStaticServlet.java — @WebServlet("/uploads/*")

2.3 上传流程

1. 接收 multipart form: kind (UploadKind), file (InputStream + ContentDisposition + BodyPart)
 2. MediaContentTypePolicy.allows(kind, contentType) → 类型检查
 3. UploadSizePolicy.allowsReportedSize(kind, sizeBytes) → 报告大小检查
 4. 文件名脱敏: Path.of(originalName).getFileName() (防目录遍历)
 5. 扩展名映射: audio类型按MIME映射 → 兜底取原始扩展名
 6. 存储: <upload.root>/<kind.lowercase>/<UUID>.<ext>
 7. UploadSizePolicy.allowsStoredSize(kind, storedSize) → 实际大小严格检查
 8. 超过限制 → 删除文件 + 返回错误
 9. INSERT INTO media_files (owner_id, kind, original_name, stored_name, url, content_type, size_bytes)
 10. 返回 MediaFile 记录

代码位置：src/main/java/com/example/chat/media/MediaService.java — save() (72行)

2.4 UploadKind 枚举 — 上传类别与大小限制

枚举值	最大	用途
CHAT_IMAGE	5 MB	聊天图片
MOMENT_IMAGE	5 MB	动态图片
MOMENT_VIDEO	50 MB	动态视频
VOICE_MESSAGE	10 MB	语音消息
AVATAR	2 MB	用户头像
BACKGROUND	5 MB	个人/群背景图

代码位置：src/main/java/com/example/chat/media/UploadKind.java

2.5 MediaContentTypePolicy — 允许的MIME类型

UploadKind	允许类型
CHAT_IMAGE, MOMENT_IMAGE, AVATAR, BACKGROUND	image/jpeg, image/png, image/gif, image/webp
MOMENT_VIDEO	video/mp4, video/webm
VOICE_MESSAGE	audio/webm, audio/ogg, audio/mpeg, audio/mp4, audio/wav, audio/x-wav

代码位置：[src/main/java/com/example/chat/media/MediaContentTypePolicy.java](#)

2.6 UploadStaticServlet — Range 支持

```
GET /uploads/images/uuid.jpg
  → 路径遍历检测 (拒绝 "..")
  → 文件存在检查
  → 解析 Range header: "bytes=START-END" 或 "bytes=-SUFFIX"
  → Range有效 → 206 Partial Content (LimitedOutputStream 精确截断)
  → 无Range → 200 Full Content
```

代码位置：[src/main/java/com/example/chat/media/UploadStaticServlet.java](#)

2.7 MediaUrlBuilder — URL前缀动态替换

```
// build(): 构建完整URL
//   → <publicBaseUrl>/uploads/<kind>/<storedName>

// normalize(): 重写已存储URL的前缀
//   → 如果URL包含 "/uploads/" → 替换前缀
//   → 如果URL以 http:// 或 https:// 开头 → 保持不变 (绝对URL)
//   → 否则 → 前置 publicBaseUrl
```

代码位置：[src/main/java/com/example/chat/media/MediaUrlBuilder.java](#)

三、数据库设计

3.1 media_files 表

列名	类型	约束	说明
id	BIGINT	PK, AUTO_INCREMENT	媒体ID
owner_id	BIGINT	FK → users, NOT NULL	上传者
kind	VARCHAR(32)	NOT NULL	UploadKind枚举名
original_name	VARCHAR(180)	NOT NULL	原始文件名
stored_name	VARCHAR(180)	NOT NULL	UUID存储名
url	VARCHAR(255)	NOT NULL	访问URL
content_type	VARCHAR(120)	NOT NULL	MIME类型
size_bytes	BIGINT	NOT NULL	文件大小(字节)
created_at	TIMESTAMP	NOT NULL	上传时间

建表SQL：[src/main/resources/schema.sql](#) 第167-178行

四、前端设计

4.1 图片上传

前端通过 `uploadChatImage(file)` 函数处理：

```
async function uploadChatImage(file) {
  const formData = new FormData();
  formData.append("kind", "CHAT_IMAGE");
  formData.append("file", file);
  const result = await ChatApi.request("/media", {
    method: "POST",
    body: formData,
  });
  return result; // 返回 MediaFile 对象
}
```

代码位置：`src/main/webapp/assets/js/chat.js` — `uploadChatImage()` (~1137行)

4.2 语音消息录制

使用 `MediaRecorder` API：

```
async function startVoiceRecording() {
  const stream = await navigator.mediaDevices.getUserMedia({audio: true});
  voiceRecorder = new MediaRecorder(stream, {
    mimeType: preferredVoiceMimeType() // 编解码器检测
  });
  voiceRecorder.ondataavailable = e => voiceChunks.push(e.data);
  voiceRecorder.onstop = async () => {
    const blob = new Blob(voiceChunks, {type: voiceRecorder.mimeType});
    const media = await uploadVoiceMessage(blob); // → POST /api/media with
kind=VOICE_MESSAGE
    sendMessage({type: "VOICE", content: media.url, mediaId: media.id});
  };
  voiceRecorder.start();
}
```

代码位置：

- `startVoiceRecording()`： `src/main/webapp/assets/js/chat.js` (~1225行)
- `stopVoiceRecording()`： `src/main/webapp/assets/js/chat.js` (~1281行)
- `uploadVoiceMessage()`： `src/main/webapp/assets/js/chat.js` (~1151行)
- `preferredVoiceMimeType()`： `src/main/webapp/assets/js/chat.js` (~1167行)